

Baseline Models

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

March 20, 2024

Contents

01 The Forecasting Process

02 Decomposition Of A Time Series

03 Baseline Models

04 Forecasting Method Evaluation

05 Forecasting The Historical Mean

06 Forecasting The Last Year's Mean

07 Forecasting The Last Known Value

08 Naive Seasonal Forecast

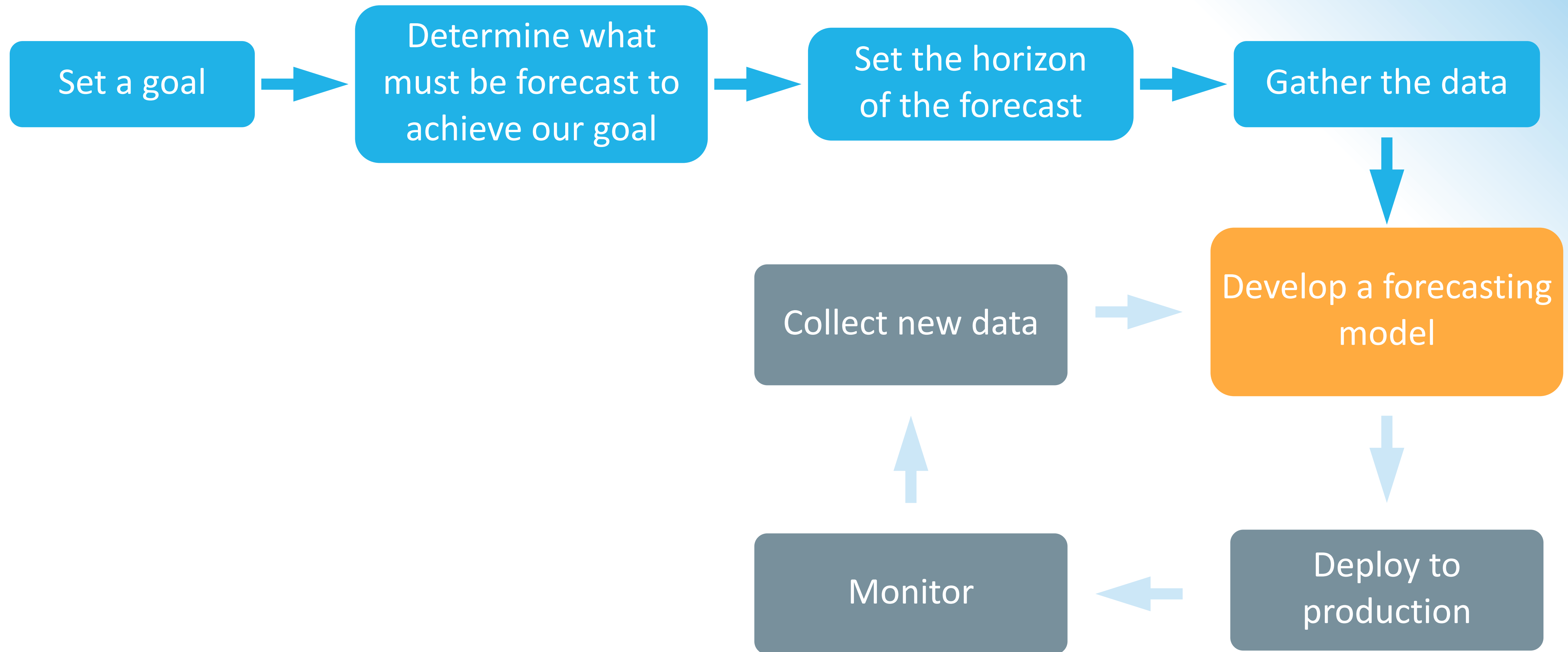
01

The Forecasting Process

Forecasting

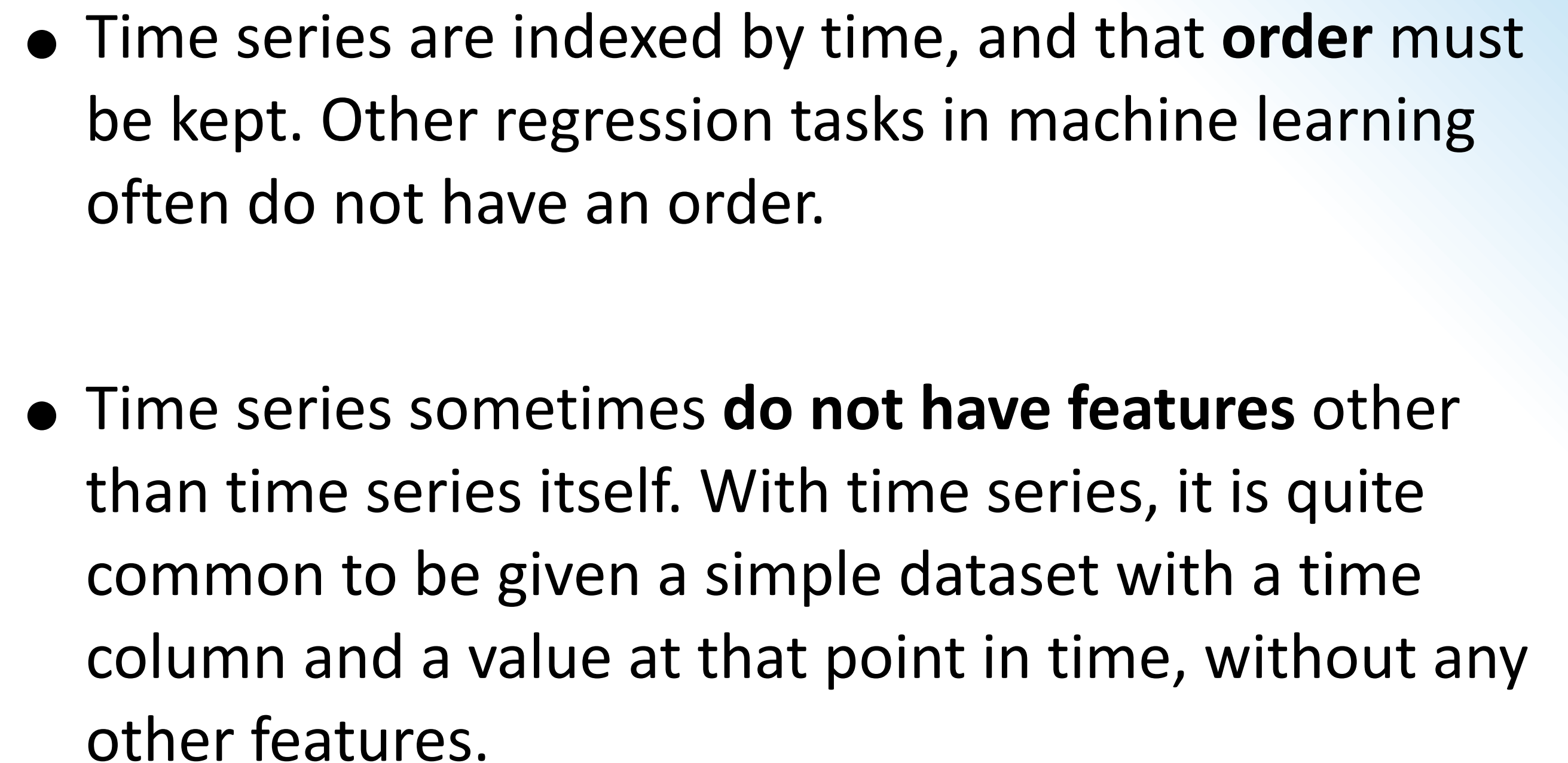
Forecasting is predicting the future using historical data and knowledge of future events that might affect our forecasts.

The Forecasting Process



How time series forecasting is different from other regression tasks?

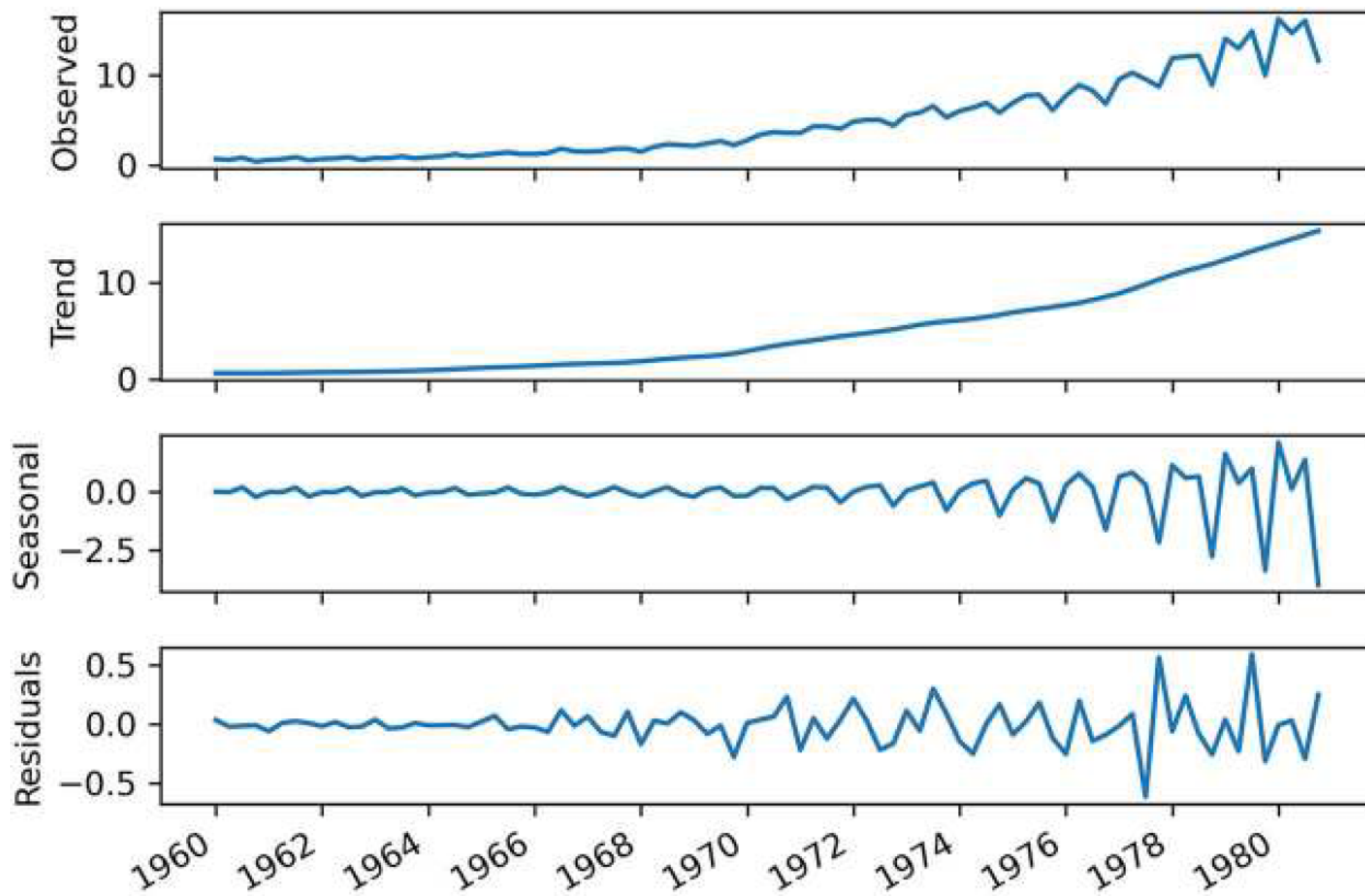
- Time series forecasting *seems* like a typical regression problem: we have some historical data, and we wish to build a mathematical expression that will express future values as a function of past values.
- However, there are some key *differences* between time series forecasting and regression for time-independent scenarios.



02

Decomposition Of A Time Series

^{def} What is a Decomposition?



- **Decomposition** of a time series is a statistical task that separates a time series into its different components:

trend, seasonal component and residuals.
- We can visualise each individual component, which will help us identify the trend and seasonal pattern in the data, which is not always straightforward just by looking at a dataset.

^{def} Trend, Seasonal, Residuals

Trend

- ◆ The **trend** is the slow-moving changes in a time series.
- ◆ Think of it as a line through most of the data points to show the general direction of a time series.

Decomposition

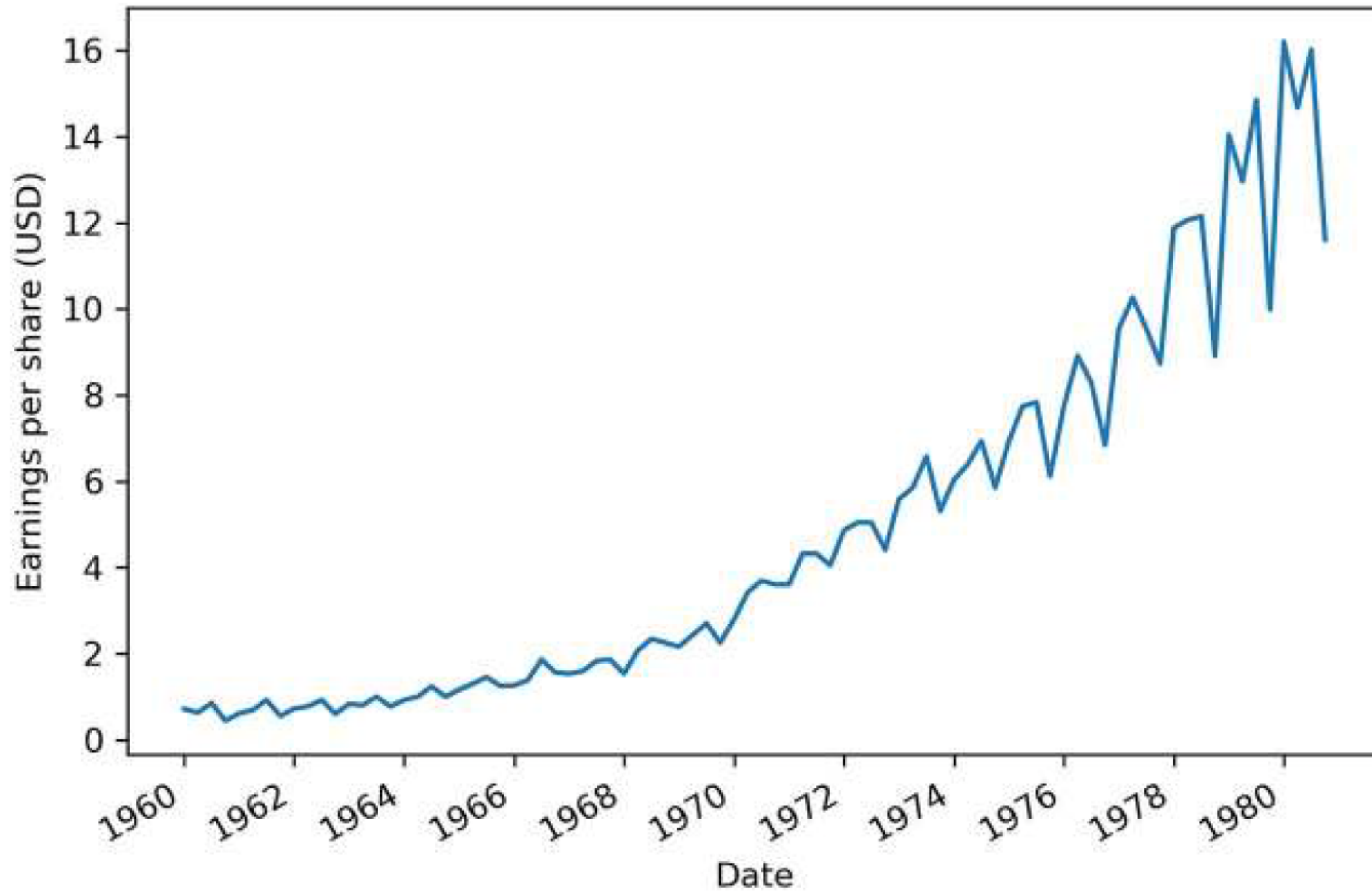
Seasonal

- ◆ The **seasonal component** captures the seasonal variation, which is a cycle that repeatedly occurs over a fixed period of time.
- ◆ It shows how we deviate from the trend.

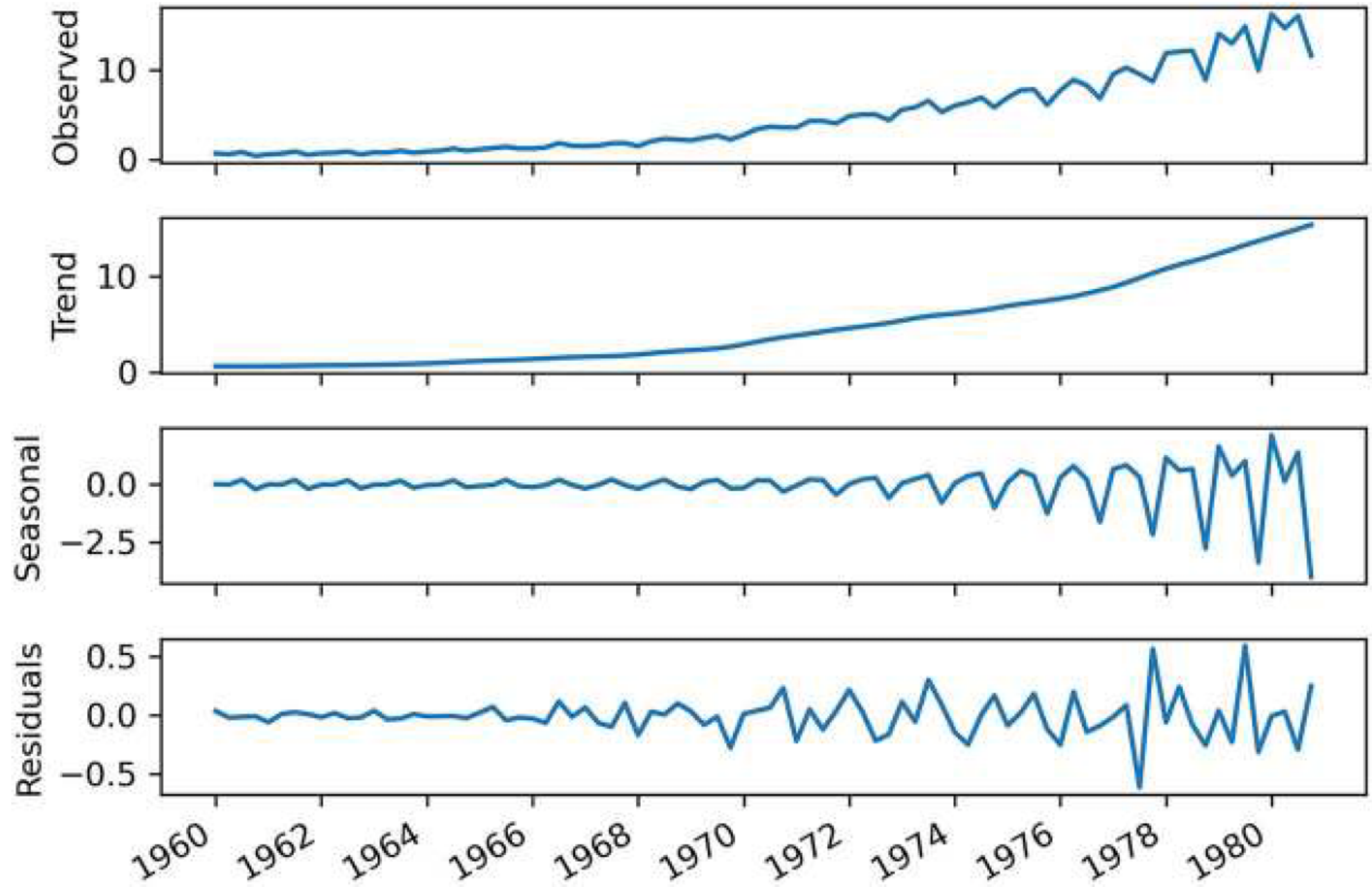
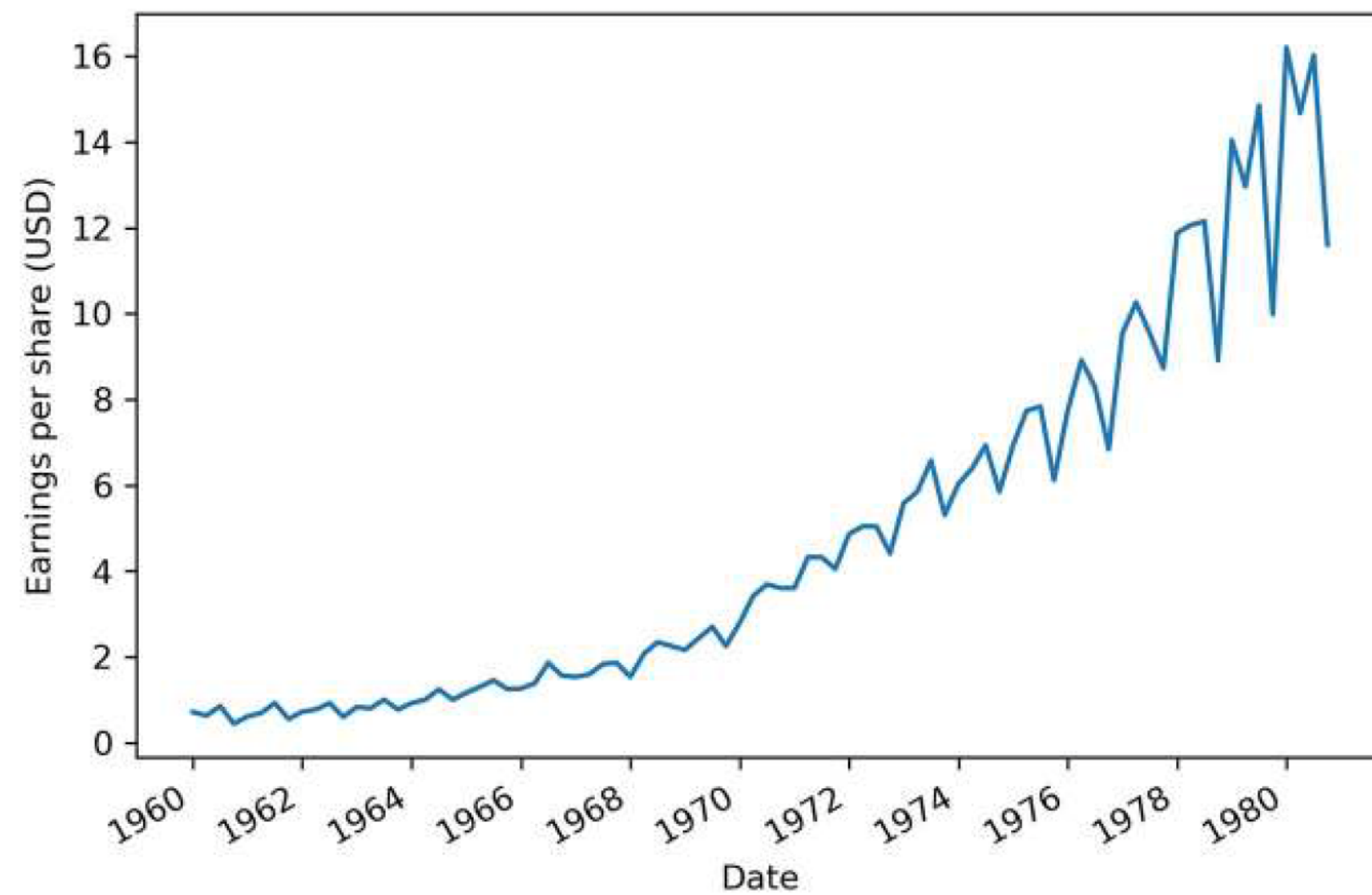
Residuals

- ◆ The **residuals** is what cannot be explained by either the trend or the seasonal components.
- ◆ Think of the them as adding the Trend and Seasonal graphs together and comparing the value at each point in time to the Observed graph.
- ◆ Residuals usually correspond to random errors, also termed *white noise* — information that we cannot model or predict.

Example 1: Quarterly earnings per share of Johnson & Johnson from 1960 to 1980



Example 1: Quarterly earnings per share of Johnson & Johnson from 1960 to 1980



03

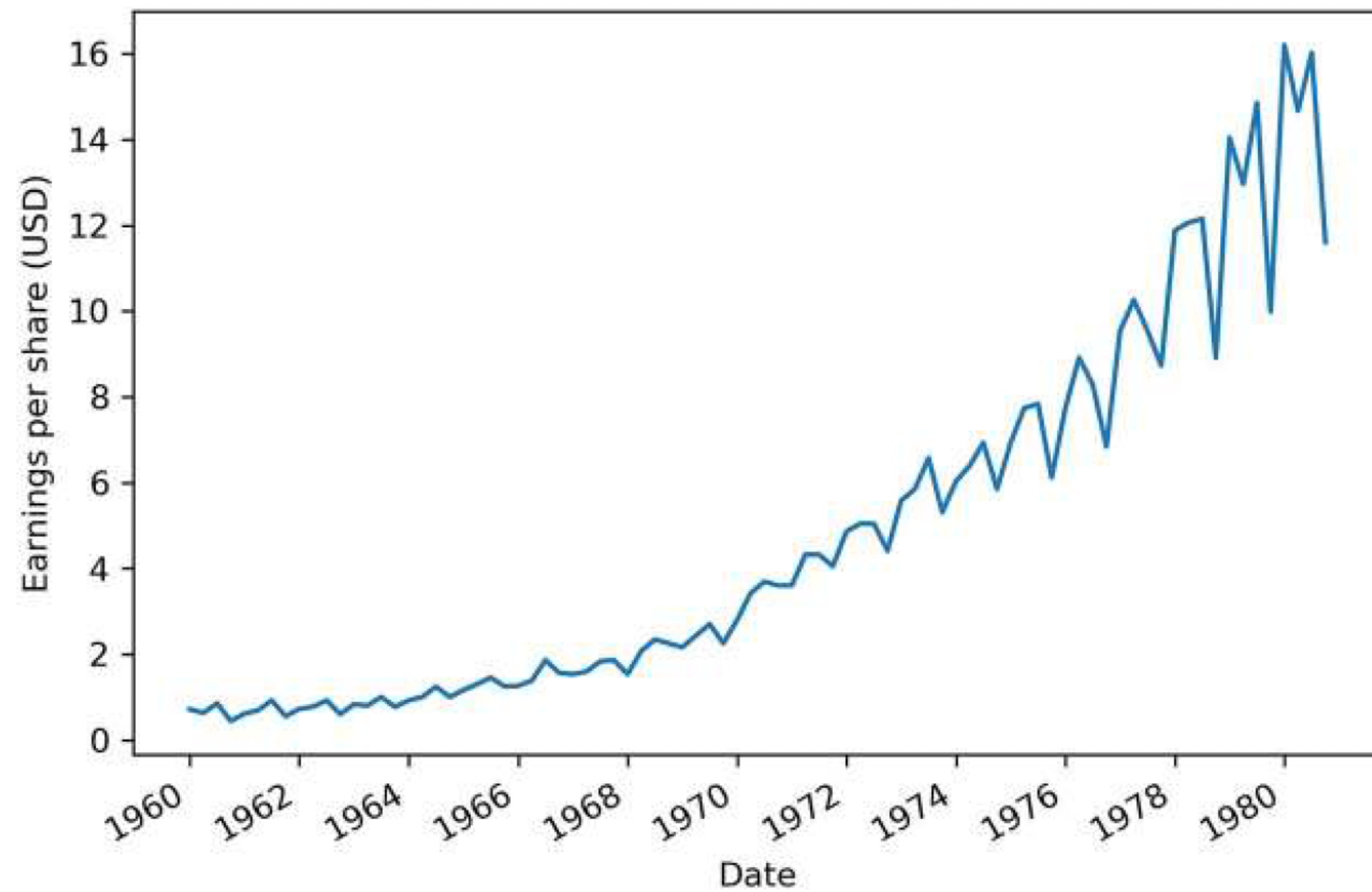
Baseline Models

def What is a Baseline Model?



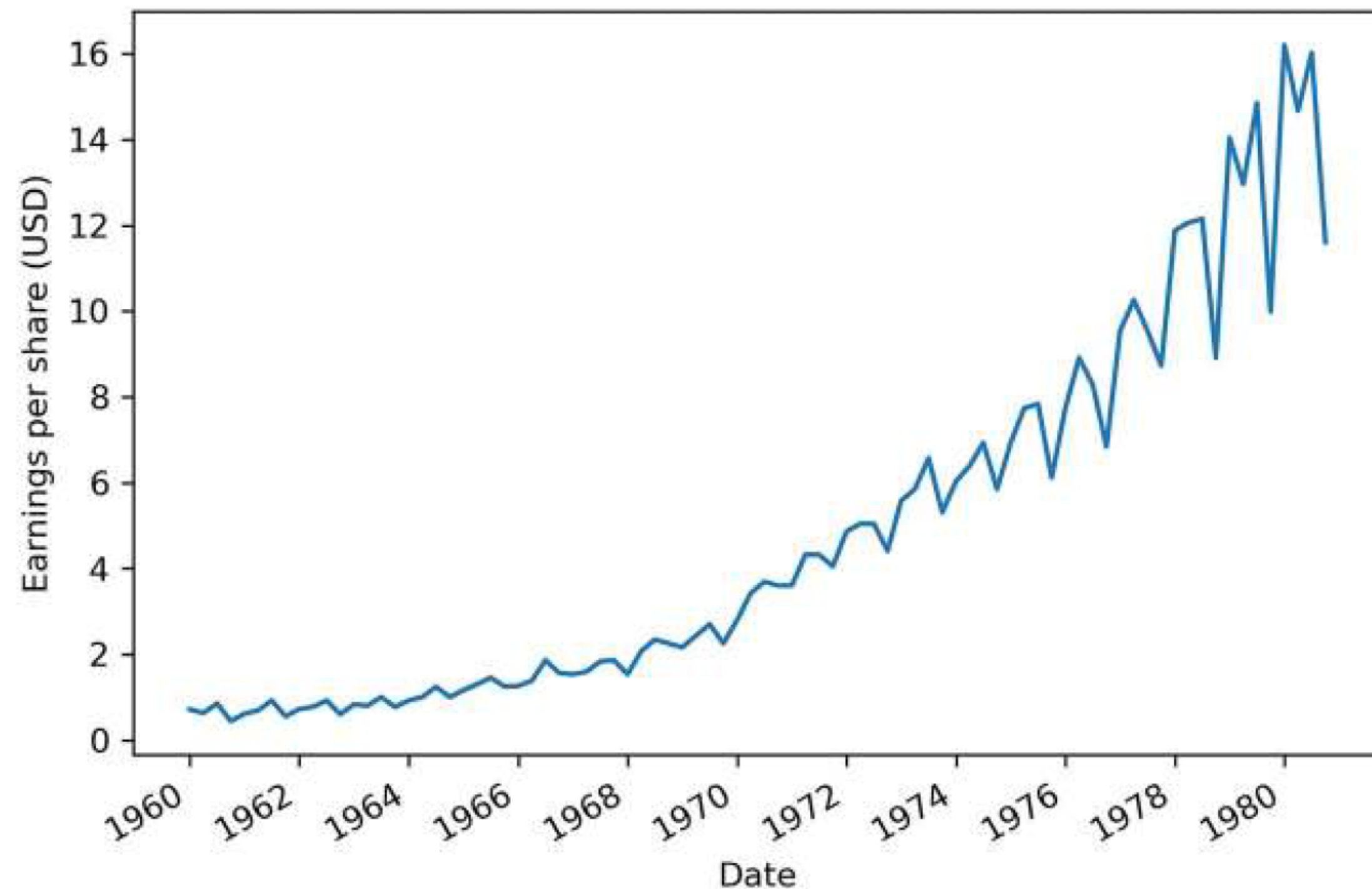
- A **baseline model** is a trivial solution to our problem.
- It often uses heuristics, or simple statistics, to generate predictions.
- The baseline model is the simplest solution you can think of — it should not require any training, and the cost of implementation should be very low.

Example 2: Quarterly earnings per share of Johnson & Johnson for 1981



- If we want to predict the EPS of Johnson & Johnson for the year 1981, what would be the most simple and naive forecast?

Example 2: Quarterly earnings per share of Johnson & Johnson for 1981



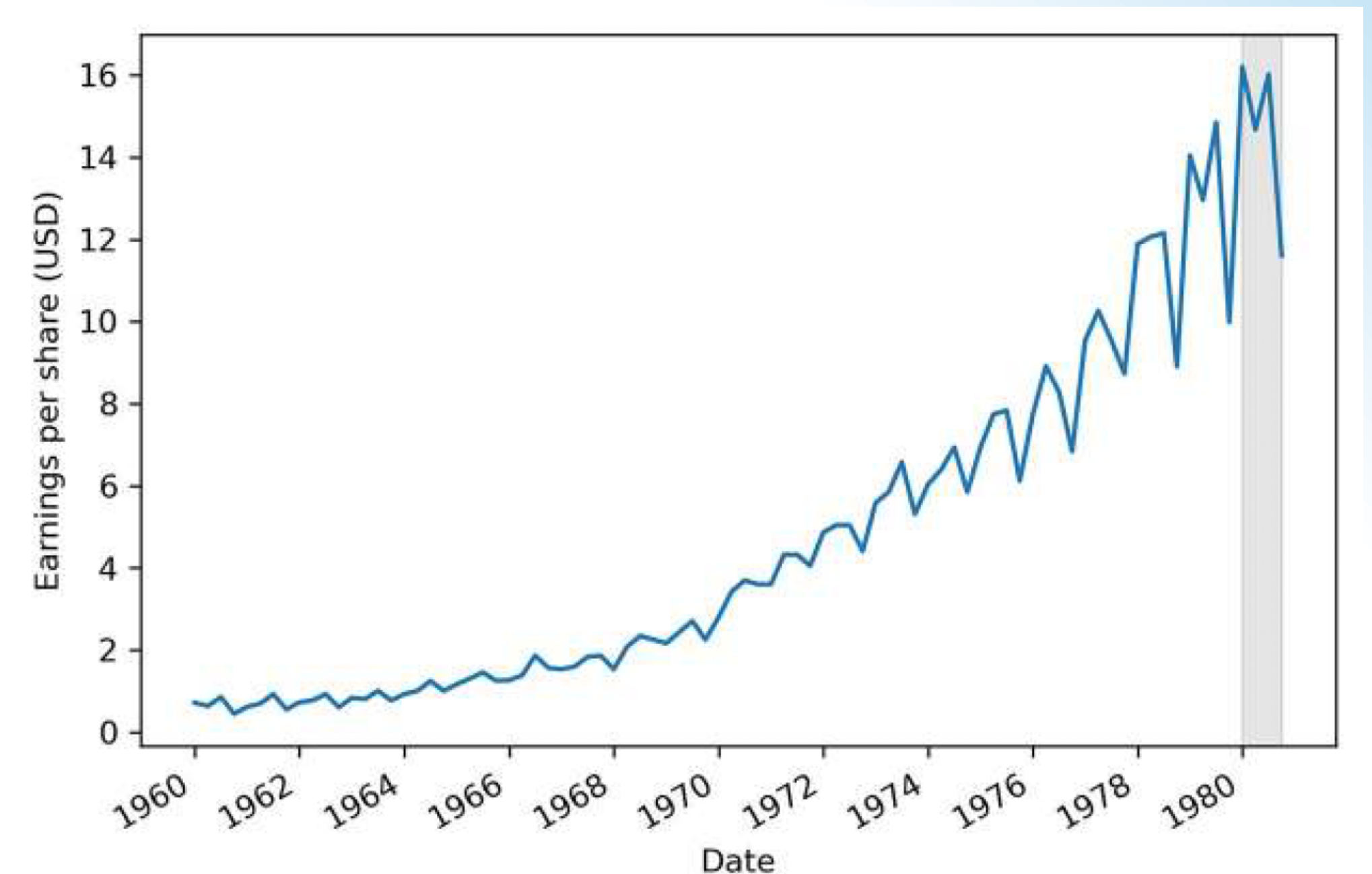
- Basic Solution 1: Compute the mean of the values over a certain period and assume that future values will be equal to that mean.
- Basic Solution 2: Forecast the last recorded data point.
- Basic Solution 3: If there is a cyclical pattern in our data, then simply repeat that pattern into the future.

04

How well does a method forecast the future?

Train and Test

- We can check how a method is good by testing how it predicts on the sample that we have. For that, split the data that we have into a **train set** for training and a **test set** for testing.
- In the example of Johnson & Johnson, given that our horizon is 1 year, our *train set* will start in 1960 and go all the way to the end of 1979. We will save the data collected in 1980 for our *test set*.
- That way we can measure the performance of our models and see how they would perform when we forecast beyond the data we have, which in this case is 1981 and later.



Prediction Accuracy Measures

The **forecast error** for time period t , denoted e_t , is defined as the difference between the actual value (y_t) and the forecast value (F_t):

$$e_t = y_t - F_t.$$

MAE or **MAD** (mean absolute error/deviation):

$$\frac{1}{n} \sum_{t=1}^n |e_t|$$

This gives the magnitude of the average absolute error.

Average error:

$$\frac{1}{n} \sum_{t=1}^n e_t$$

It gives an indication of whether the forecasts are on average over- or under-predicting.

MAPE (mean absolute percentage error):

$$\frac{1}{n} \sum_{t=1}^n \left| \frac{e_t}{y_t} \right| \times 100$$

It gives a percentage score of how forecasts deviate (on average) from the actual values.

RMSE (root mean squared error):

$$\sqrt{\frac{1}{n} \sum_{t=1}^n e_t^2}$$

It measures the square root of the average squared differences between forecasted values and actual values, providing a comprehensive view of prediction accuracy while considering both magnitude and direction of errors.

05

Forecasting The Historical Mean

Step 1: Setup

- Load the dataset. Use the **pandas** library and load the dataset into a **DataFrame** using the **read_csv** method.

```
import pandas as pd  
df = pd.read_csv('../data/jj.csv')
```

- A **DataFrame** is the most-used data structure in pandas. It is a 2D labeled data structure with columns that can hold different types of data, such as strings, integers, floats, or dates.

- To display the first five and the last five entries:

```
df.head()  
df.tail()
```

- Split the dataset into the **train** and **test sets**. The train set will consist of the data from 1960 to the end of 1979, and the test set will consist of the four quarters of 1980.

```
train = df[:-4]  
test = df[-4:]
```

	date	data
0	1960-01-01	0.71
1	1960-04-01	0.63
2	1960-07-02	0.85
3	1960-10-01	0.44
4	1961-01-01	0.61

	date	data
79	1979-10-01	9.99
80	1980-01-01	16.20
81	1980-04-01	14.67
82	1980-07-02	16.02
83	1980-10-01	11.61

Step 2: Implementation

- Compute the historical mean. Use the **numpy** library.

```
import numpy as np  
historical_mean = np.mean(train['data'])  
print(historical_mean) # this gives 4.31 USD
```

This means that from 1960 to the end of 1979, the quarterly EPS of Johnson & Johnson is on average 4.31 USD.

- Set the historical mean as a forecast:

```
test.loc[:, 'pred_mean'] = historical_mean
```

- Calculate the mean absolute percentage error (MAPE):

```
def mape(y_true, y_pred):  
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
mape_hist_mean = mape(test['data'], test['pred_mean'])  
print(mape_hist_mean)
```

Running the function gives a MAPE of 70.00%.

	date	data
0	1960-01-01	0.71
1	1960-04-01	0.63
2	1960-07-02	0.85
3	1960-10-01	0.44
4	1961-01-01	0.61

	date	data
79	1979-10-01	9.99
80	1980-01-01	16.20
81	1980-04-01	14.67
82	1980-07-02	16.02
83	1980-10-01	11.61

Step 3: Visualisation

- Use the **matplotlib** library.

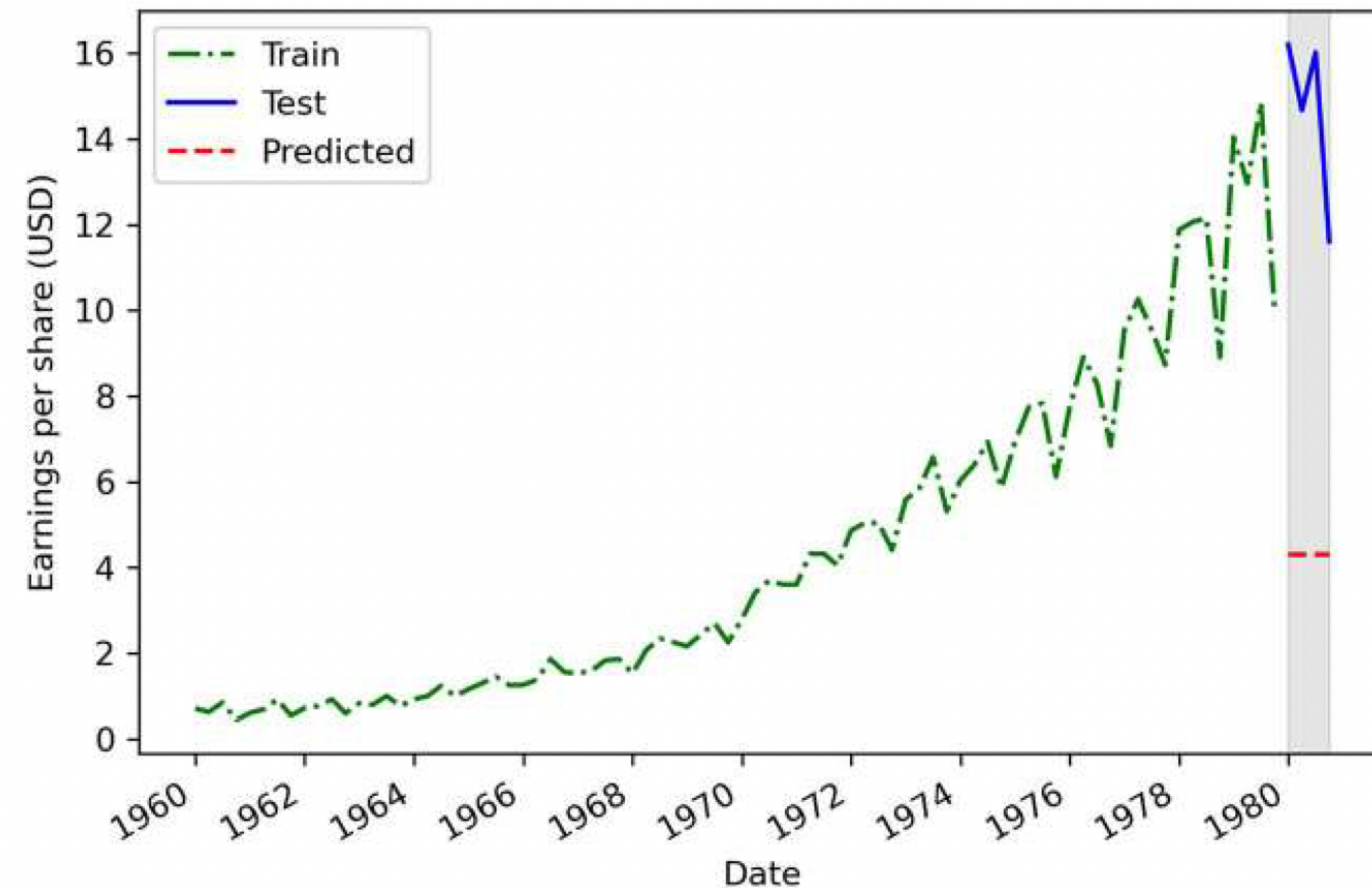
```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots()
```

```
ax.plot(train['date'], train['data'], 'g-.', label='Train')  
ax.plot(test['date'], test['data'], 'b-', label='Test')  
ax.plot(test['date'], test['pred_mean'], 'r--', label='Predicted')  
ax.set_xlabel('Date')  
ax.set_ylabel('Earnings per share (USD)')  
ax.axvspan(80, 83, color='#808080', alpha=0.2)  
ax.legend(loc=2)
```

```
plt.xticks(np.arange(0, 85, 8), [1960, 1962, 1964, 1966, 1968, 1970, 1972,  
1974, 1976, 1978, 1980])
```

```
fig.autofmt_xdate() # automatically format the tick labels  
plt.tight_layout() # remove excess white space around the  
figure
```



06

Forecasting The Last Year's Mean

Implementation

- Compute the last year's mean. Use the **numpy** library.

```
last_year_mean = np.mean(train.data[-4:])  
print(last_year_mean) # this gives $12.96 USD
```

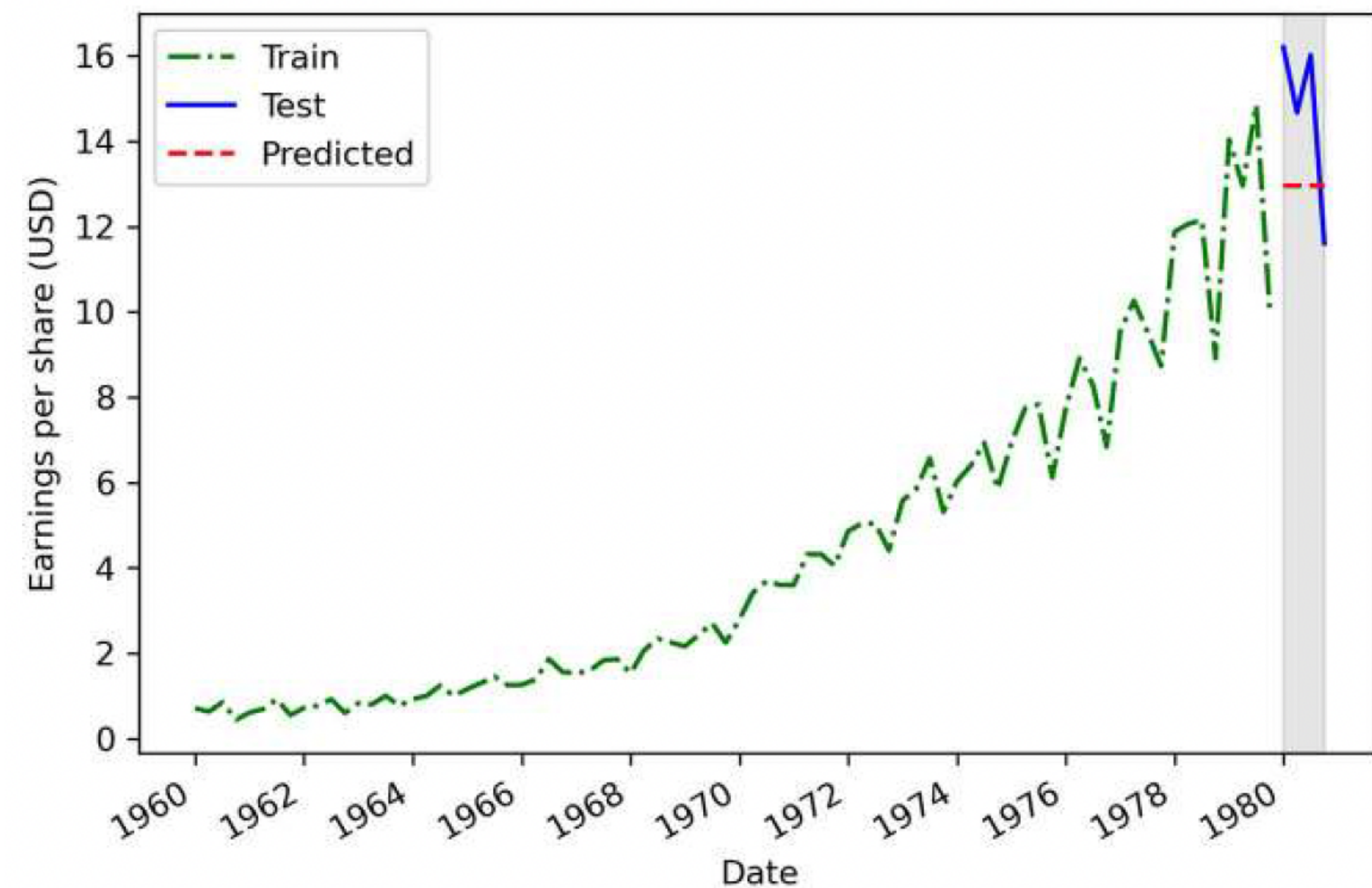
- Set the last year's mean as a forecast:

```
test.loc[:, 'pred__last_yr_mean'] = last_year_mean
```

- Calculate the mean absolute percentage error (MAPE):

```
mape_last_year_mean = mape(test['data'], test['pred__last_yr_mean'])  
print(mape_last_year_mean)
```

Running the function gives a MAPE of 15.60%, which is better than the previous one. We can learn from this baseline that future values likely depend on past values that are not too far back in history. This is a sign of *autocorrelation*.



07

Forecasting The Last Known Value

Implementation

- Retrieve the last known value.

```
last = train.data.iloc[-1]  
print(last) # this gives $9.99 USD
```

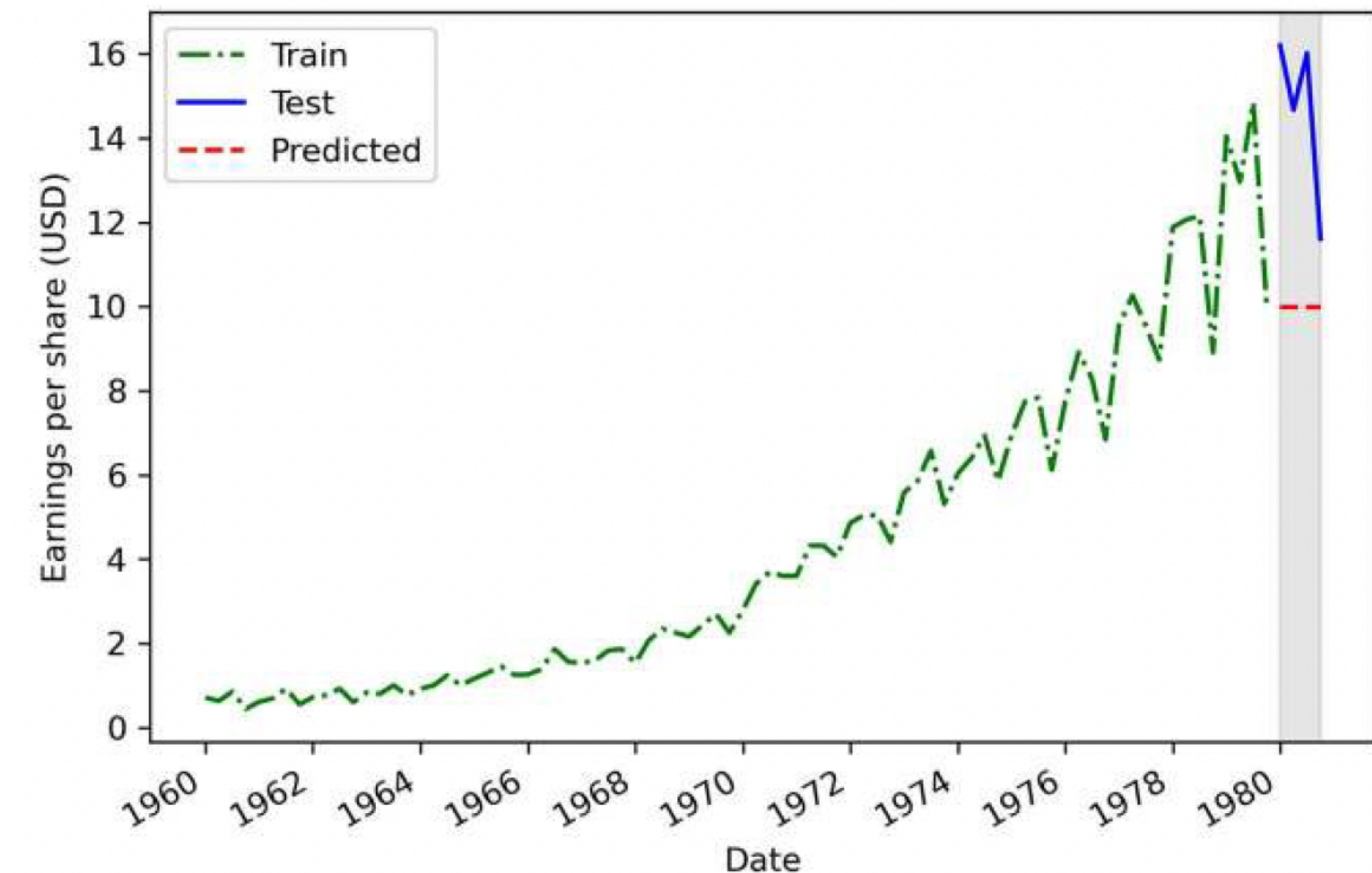
- Set the last known value as a forecast:

```
test.loc[:, 'pred_last'] = last
```

- Calculate the mean absolute percentage error (MAPE):

```
mape_last = mape(test['data'], test['pred_last'])  
print(mape_last)
```

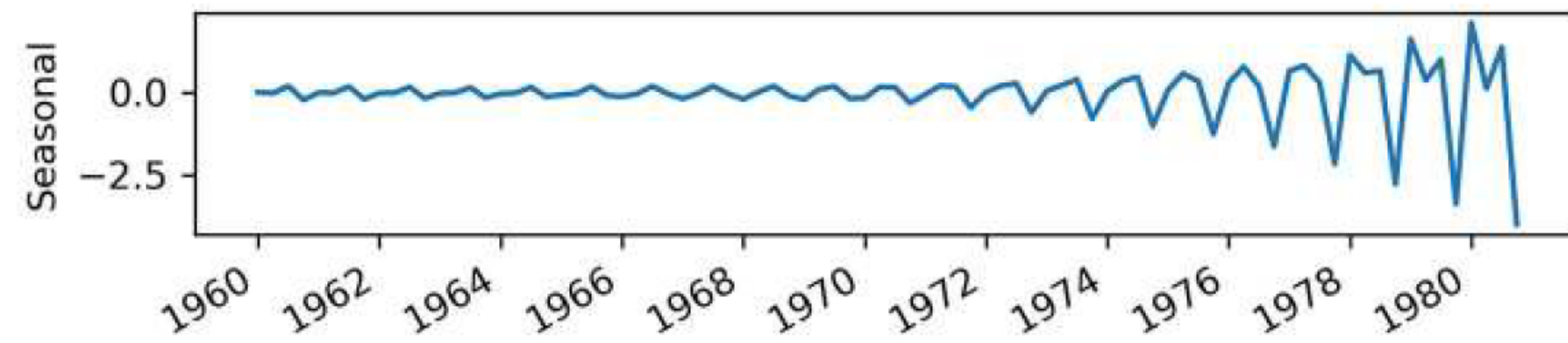
Running the function gives a MAPE of 30.45%, which is worse than the previous one. This can be explained by the fact that the EPS displays a cyclical behaviour, where it is high during the first three quarters and then falls at the last quarter.



08

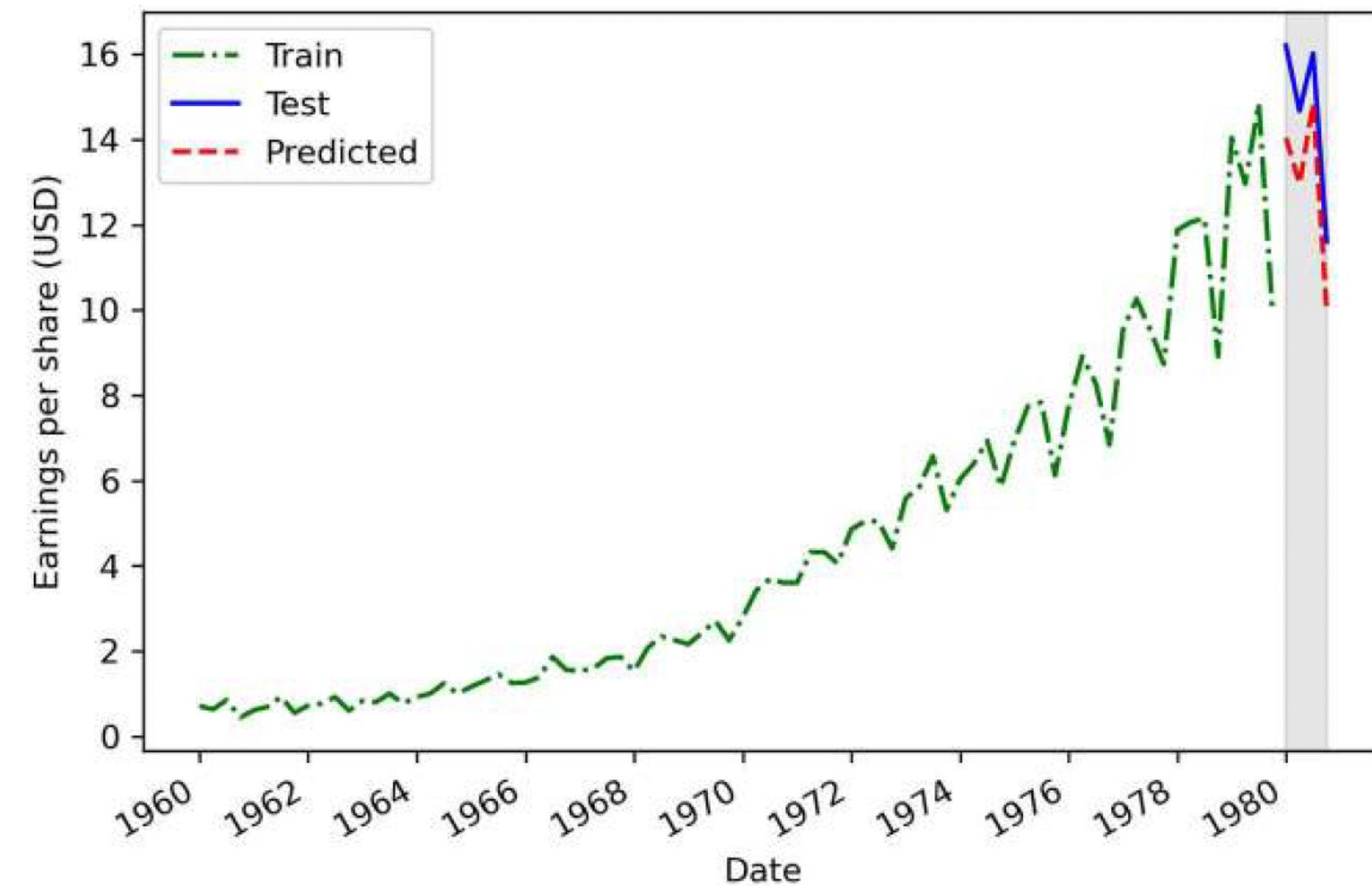
Naive Seasonal Forecast

Implementation



- Our predictions are the last four values of our train set, which correspond to the quarters of 1979:

```
test.loc[:, 'pred_last_season'] = train['data'][-4:].values
```

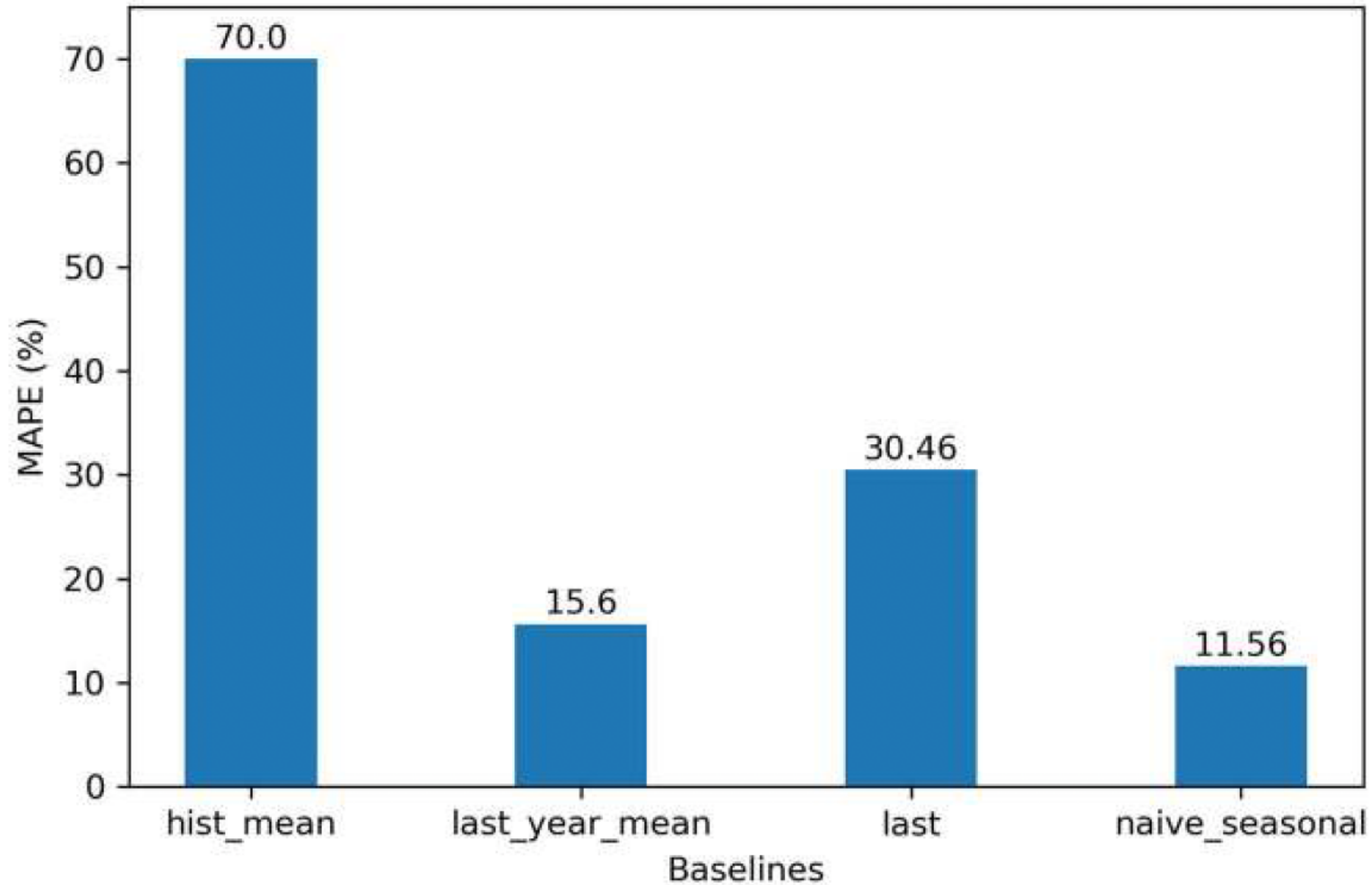


- Calculate the mean absolute percentage error (MAPE):

```
mape_naive_seasonal = mape(test['data'], test['pred_last_season'])  
print(mape_naive_seasonal)
```

Running the function gives a MAPE of **11.56%**, which is better than the previous one. This means that seasonality has a significant impact on future values, since repeating the last season into the future yields fairly accurate forecasts.

Which model is better?



What is next?

- A baseline model serves as *a basis for comparison*. We will develop more complex models by applying statistical learning or deep learning techniques, and when we evaluate our more complex solutions against the test set and record our error metrics, we can compare them to those of the baseline.
- There will be special situations in which a time series can only be forecast using naive methods. These are special cases where the process moves at random and cannot be predicted using statistical learning methods. This means that we are in the presence of *a random walk* — we'll examine this during the next lesson.

Thank you!